

Replace Scoring with Arrangement: A Contextual Set-to-Arrangement Framework for Learning-to-Rank

Jiarui Jin¹, Xianyu Chen¹, Weinan Zhang¹, Mengyue Yang², Yang Wang³, Yali Du⁴,
Yong Yu¹, Jun Wang²

¹Shanghai Jiao Tong University, ²University College London, ³East China Normal University, ⁴King's College London
{jinjiarui97,xianyujun,wnzhang,yyu}@sjtu.edu.cn,{mengyue.yang,20,jun.wang}@ucl.ac.uk,ywang@sei.ecnu.edu.cn,
yali.du@kcl.ac.uk

ABSTRACT

Learning-to-rank is a core technique in the top-N recommendation task, where an ideal ranker would be a mapping from an item set to an arrangement (a.k.a. permutation). Most existing solutions fall in the paradigm of probabilistic ranking principle (PRP), i.e., first score each item in the candidate set and then perform a sort operation to generate the top ranking list. However, these approaches neglect the contextual dependence among candidate items during individual scoring, and the sort operation is non-differentiable. To bypass the above issues, we propose Set-To-Arrangement Ranking (STARank), a new framework directly generates the permutations of the candidate items without the need for individually scoring and sort operations; and is end-to-end differentiable. As a result, STARank can operate when only the ground-truth permutations are accessible without requiring access to the ground-truth relevance scores for items. For this purpose, STARank first reads the candidate items in the context of the user browsing history, whose representations are fed into a Plackett-Luce module to arrange the given items into a list. To effectively utilize the given ground-truth permutations for supervising STARank, we leverage the internal consistency property of Plackett-Luce models to derive a computationally efficient list-wise loss. Experimental comparisons against 9 the state-of-the-art methods on 2 learning-to-rank benchmark datasets and 3 top-N real-world recommendation datasets demonstrate the superiority of STARank in terms of conventional ranking metrics. Notice that these ranking metrics do not consider the effects of the contextual dependence among the items in the list, we design a new family of simulation-based ranking metrics, where existing metrics can be regarded as special cases. STARank can consistently achieve better performance in terms of PBM and UBM simulation-based metrics.

KEYWORDS

Learning-to-Rank, Contextual Set-to-Arrangement, Efficient Supervision Generation

1 INTRODUCTION

Learning-to-rank (LTR) covers a branch of machine learning methods for optimizing ranking performance and is a core technique for search engine and top-N recommendation applications [24]. Conventional LTR algorithms are usually designed on the basis of the probability ranking principle (PRP) [32], which consists of a scoring function that assigns an individual score to each item, and subsequently produces the ranking list by sorting items according to their assigned scores. Note that the crucial difference between LTR and classical machine learning tasks (e.g., regression, classification) is that in LTR, the exact score of each item is not important, but the relative orders of the items matter. It is supported by the behavior analysis results on search engines [21, 38], which manifests that user interactions show strong comparison patterns, namely users usually compare multiple items before generating a click action.

Being aware of PRP's limitation of independently assigning the ranking score for each item and using non-differentiable sort operations [37], there are roughly two lines of previous researches. One direction [1, 3, 25] investigates incorporating context into the ranking process and assigning scores to candidate items while taking into account their dependencies within the context. The other direction [6, 14, 34, 37] designs a surrogate differentiable list-wise loss based on permutation probabilities proposed in [26] to avoid the use of the sort operation. However, all these methods still fall into a paradigm of first individually scoring items and then putting them into correct positions in descending order of their scores.

In this paper, we propose a new Set-To-Arrangement Ranking (STARank) framework, which can directly generate the permutations of candidate items without the need for individually scoring and sort operations. Hence, our proposed framework is end-to-end differentiable, and can be directly supervised with ground-truth permutations instead of ground-truth scores for all candidate items. Concretely, as depicted in Figure 1(a), the ultimate goal is to arrange the candidate items into a permutation close to the ground-truth permutation based on the historical records from users. For this purpose, we develop a novel read-arrange-supervise paradigm.

Our reader component, as depicted in Figure 1(b), is required to encode two types of input items, namely, the user browsed items and the candidate items. Their major difference lies in that the user browsed items are characterized by their permutation orders, while the candidate items do not have an inherent permutation order. Therefore, we first build a permutation-sensitive module to encode the browsed items, whose outputs are further fed into a permutation-invariant module to produce the representation of each candidate item. Then, our arranger, as shown in Figure 1(c),

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Pre-print, Under review, 2023

© 2023 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

applies a Plackett-Luce (PL) module to match the candidate items to the remaining positions.

Consider that we only have access to the ground-truth permutations. We leverage the internal consistency property of the arranger (i.e., PL models) to derive a list-wise loss function, as shown in Figure 1(d). Compared to existing loss functions based on the ground-truth relevance scores, our loss focuses on identifying and correcting the relative permutations of neighboring pairs that have errors during arranging. In other words, this list-wise loss is typically designed for our recursive permutation generation. Our theoretical analysis also reveals that the proposed framework is permutation-invariant regarding the candidate items (namely any permutation of the input items would not change the output ranking results).

In addition, we notice that conventional ranking metrics overlook the effects of contextual dependence among the items (i.e., the probability of a user favoring an item is affected by other items placed in the same list). Therefore, we develop a new series of simulation (i.e., click models) based metrics, where NDCG can be regarded as special cases of using PBM simulations [31].

We conduct experiments on 2 learning-to-rank benchmark datasets and 3 real-world recommendation datasets. Empirical results demonstrate that STARank can consistently outperform 9 the state-of-art baseline methods in terms of both conventional ranking metrics and new proposed simulation-based metrics.

2 PRELIMINARIES

2.1 Problem Formulation

LTR algorithms usually assume that each item d has its utility in the context of a user (or a query) q ¹, which is often modelled as the relevance score between q and d . Let $\mathcal{D}$ denote the set of candidate items and $\mathcal{D}_q$ denote the set of candidate items associated with q . Besides these candidate items to be arranged (a.k.a., permuted), we would also have access to user q 's historical records, a sequence of the browsed item list, denoted as $\mathcal{H}_q$. Notably, $\mathcal{D}_q$ and $\mathcal{H}_q$ are subsets of $\mathcal{D}$ (i.e., $\mathcal{D}_q \subseteq \mathcal{D}$, $\mathcal{H}_q \subseteq \mathcal{D}$), and there exists a permutation order in each $\mathcal{H}_q$ but no permutation order in each $\mathcal{D}_q$. We further use r_d and r_d^* to denote the predicted and ground-truth relevance of d respectively.

Prevailing ranking methods formulate the ranking problem as to predict r for each item, whose risk function is defined as

$$\mathcal{F}_{\text{point}} = \sum_q \sum_{d \in \mathcal{D}_q} \ell_{\text{point}}(f_{\text{point}}(d|\mathcal{H}_q), r_d^*) \quad (1)$$

where $\ell_{\text{point}}(\cdot, \cdot)$ denotes a point-wise loss function, $f_{\text{point}}(d|\mathcal{H}_q)$ is a ranker that makes prediction over one item d in the context of $\mathcal{H}_q$. In practice, following PRP, the trained ranker is used to assign an individual score for each item independently and sort the items according to their scores. However, this pipeline internally suffers from the following limitations: (i) it lacks the consideration over the contextual dependence among items, as the score of each item is solely determined by itself; (ii) it requires the sort operation, which brings the non-differentiable issue.

¹In the context of LTR [24], the terms "query" and "user" normally appear in the search engines and the top-N recommendation scenarios respectively. For simplicity, we mainly use "user" in this paper.

Table 1: A summary of notations associated with π .

Notations	Explanations
$\pi_i \in \mathcal{D}_q$	The item at the i -th position
$\pi_{<i} \subseteq \mathcal{D}_q$	A set of the items at the higher positions than i

Instead, we formulate the ranking problem as to seek for the ground-truth permutation for the given set of candidate items. Formally, let $\pi(q)$ and $\pi^*(q)$ denote the predicted permutation and ground-truth permutation of the given item set $\mathcal{D}_q$ respectively. In other words, $\pi(q)$ is the sequence of the arranged items in $\mathcal{D}_q$. In the followings, we use π and π^* instead of $\pi(q)$ and $\pi^*(q)$ for convenience. We list the key notations associated with π in Table 1.

Our set-to-arrangement ranking problem can be described as

Definition 2.1 (Set-to-Arrangement Ranking Problem). For each user/query q , given a tuple $(\mathcal{D}_q, \mathcal{H}_q, \pi^*)$ where $\mathcal{D}_q$ is the set of candidate items and π^* is the ground-truth permutation of q , and $\mathcal{H}_q$ is the set of browsed items; our goal is to learn a ranker $f_{\text{set}}(\mathcal{D}_q|\mathcal{H}_q)$ that generates the permutation π for the given set of items $\mathcal{D}_q$ in the context of $\mathcal{H}_q$. The risk function can be written as

$$\mathcal{F}_{\text{set}} = \sum_q \ell_{\text{list}}(f_{\text{set}}(\mathcal{D}_q|\mathcal{H}_q), \pi^*), \quad (2)$$

where $\ell_{\text{list}}(\cdot, \cdot)$ is a list-wise loss function.

We illustrate our task in Figure 1(a). We note that in our task, only the ground-truth permutation π^* is accessible, while the ground-truth relevance score r_d^* for each item d is not available.

2.2 Connections to Related Work

LTR algorithms refer to a group of machine techniques designed to solve ranking problems that have been successfully applied in multiple areas including top-N recommendation [11], search engine [19, 24]. These techniques can be broadly categorized as point-wise, pair-wise, and list-wise methods regarding the computation of training losses. The point-wise methods [13] directly treat the ranking problems as classifications or regressions by taking one item as the input to predict its score. The pair-wise methods [4, 20] take a pair of items as the input and optimize their relative positions in the final ranked list. The list-wise methods [5, 6, 37] further extend the above methods by taking multiple items together and optimizing the ranking metrics. In addition, there are recently proposed models to build multi-variant scoring functions for each item [1] or design permutation-invariant encoders [23, 25] or pointer network encoders [3] for the input item set. However, all the above methods following PBP fall into a paradigm of first individually scoring items and then putting them into correct positions through a non-differentiable sort operation [37] or a surrogate loss based on permutation probability [26]. In contrast, STARank directly learns to generate the permutation of the candidate items does not involve individually scoring items and the sort operation.

Formally, $f_{\text{set}}(\cdot|\cdot)$ considers the contextual dependence, as it directly generates the permutation orders over all the items instead of assigning individual score to each item. It allows STARank to operate when only the ground-truth permutation π^* is accessible, without requiring access to the ground-truth relevance score r_d^* for each item d . This flexibility allows our framework to be applied in

scenarios where explicit relevance scores may not be available or difficult to obtain. Also, it enables $f_{\text{set}}(\cdot)$ to be fully differentiable as it does not use the sort operation.

Additionally, $\ell_{\text{list}}(f_{\text{set}}(\cdot), \pi^*)$ evaluates the difference between the predicted permutation and the ground-truth permutation instead of individual evaluation over each item. From this perspective, our paper is also related to previous metric designs [2, 8], which investigate to enable the evaluation metrics to take the effects of the contextual dependence among the candidate items into account.

3 THE STARANK FRAMEWORK

3.1 Overview

In this section, we present the architecture design of $f_{\text{set}}(\cdot)$ as follows. We first describe our reader module (as shown in Figure 1(b)) to generate the representations of the candidate items in the context of the browsed items in Section 3.2. Then, we introduce our arranger module (as shown in Figure 1(c)) to produce the permutation of the given candidate items in Section 3.3. We also answer how to derive a our list-wise loss $\ell_{\text{list}}(f_{\text{set}}(\cdot), \pi^*)$ (as shown in Figure 1(d)) in Section 3.4.

3.2 Reader Module

For each user/query q there are two types of inputs: (i) $\mathcal{D}_q$ is a set of candidate items, which is expected to be encoded in a permutation-invariant manner. In other words, any permutation of the input items should not change the output ranking. (ii) $\mathcal{H}_q$ is a sequence of browsed items that have their natural orders and thus should be encoded in a permutation-sensitive fashion. Namely, the positions of browsed items should be considered. In the light of this, we construct $f_{\text{set}}(\mathcal{D}_q | \mathcal{H}_q)$ as follows.

Let $\mathbf{x}_d$ denote the feature vector of item d . We first introduce a permutation-sensitive (PS) module, denoted as $\text{PS}(\cdot)$, to represent the user embedding in a recursive manner as

$$\mathbf{u}_q = \text{PS}(\mathbf{u}_0, \mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{H}_q|}), \quad (3)$$

where $\mathbf{u}_q$ is the embedding vector for user q . For simplicity, we regard the user profiles (e.g., gender, age) as the original user feature, denoted as $\mathbf{u}_0$ which is fed into our PS module as shown in Figure 1(b). While practical, we adopt a recurrent neural network (RNN), i.e., an LSTM [17] to form the PS module.

Given $\mathbf{u}_q$, we then deploy a permutation-invariant (PI) module, denoted as $\text{PI}(\cdot)$ to encode the candidate items in $\mathcal{D}_q$, which can be expressed as

$$\{\mathbf{h}_d | d \in \mathcal{D}_q\} = \text{PI}(\{\mathbf{x}_d | d \in \mathcal{D}_q\}, \mathbf{u}_q), \quad (4)$$

where $\mathbf{h}_d$ is the representation vector of item d . In particular, our PI module is based on an attention mechanism [35], where the representation vector of each item d is produced by

$$\mathbf{h}'_d = \mathbf{w}_1^\top \tanh(\mathbf{W}_1 \mathbf{x}_d + \mathbf{b}_1), \quad d \in \mathcal{D}_q, \quad (5)$$

$$\mathbf{h}_d = \beta_d \mathbf{h}'_d, \quad \text{where } \beta_d = \frac{\mathbf{h}'_d \mathbf{u}_q^\top}{\sum_{j \in \mathcal{D}_q} \mathbf{h}'_j \mathbf{u}_q^\top}, \quad (6)$$

where $\mathbf{w}_1, \mathbf{W}_1, \mathbf{b}_1$ are trainable matrices and vectors.

By incorporating Eqs. (3), (5) and (6) into Eq. (4), we can obtain the representation vectors of items, being aware of the representations of the user and her browsed items.

3.3 Arranger Module

With the representation vector of each input item, we use a Plackett-Luce (PL) module to generate their permutation. Formally, the PL module, denoted as $\text{PL}(\cdot)$, is defined as

$$\pi = \text{PL}(\{\mathbf{h}_d | d \in \mathcal{D}_q\}, \mathbf{u}_q). \quad (7)$$

Noticing that the original pointer network [36] using LSTM as the encoder is permutation-sensitive, we tweak the pointer network by replacing the LSTM with the aforementioned reader.

More concretely, we construct a batch of one-hot position embedding vectors $\{\mathbf{p}_i\}_{i=1}^{|\mathcal{D}_q|}$ to encode the position information. These embeddings can be obtained by applying a LSTM as the decoder.

Let π_i denote the item at the i -th position, and $\pi_{<i}$ denote a set of the items whose positions are smaller than i , namely $\pi_{<i} = \{\pi_1, \dots, \pi_{i-1}\}$. For convenience, we define $\pi_{<1}$ as $\emptyset$.

Then, the probability distribution of item d at the i -th position can be produced as

$$s_d^i = \mathbf{v}_d^i \mathbf{u}_q^\top, \quad \text{where } \mathbf{v}_d^i = \mathbf{w}_2^\top \tanh(\mathbf{W}_2 \mathbf{h}_d + \mathbf{W}_3 \mathbf{p}_i + \mathbf{b}_2), \quad (8)$$

$$P(\pi_i = d | \pi_{<i}) = \begin{cases} e^{s_d^i} / \sum_{k \in \mathcal{D}_q \setminus \pi_{<i}} e^{s_k^i}, & \text{if } d \notin \pi_{<i}, \\ 0, & \text{otherwise.} \end{cases}, \quad (9)$$

where $i = 1, \dots, |\mathcal{D}_q|, d \in \mathcal{D}_q$; s_d^i is the trainable attention score associated with placing item d at position i and $p_d^i = P(\pi_i = d | \pi_{<i})$ is the probability representing the degree to which the PL module points item d to position i . p_d^i is computed by a softmax over the remaining items and is set to 0 for items already been arranged. Once item d is placed at position i , its embedding $\mathbf{h}_d$ is fed as the input at the next step to produce the scores of position $i + 1$.

We demonstrate the arrangement process in Figure 1(c). This way can ensure the module holds the information on the items already been arranged at each position to eventually output the predicted permutation π .

As Eq. (9) shows the step-by-step generation of π , we follow Eq. (10) in [18] to derive the predicted permutation π as

$$P(\pi | \mathcal{D}_q) = \prod_{i=1}^{|\mathcal{D}_q|} \frac{e^{s_{\pi_i}^i}}{\sum_{j=1}^{|\mathcal{D}_q|} e^{s_j^i}}, \quad (10)$$

where $i = 1, \dots, |\mathcal{D}_q|$ and $s_{\pi_i}^i$ is computed by Eq. (8). We apply Eq. (10) as $\text{PL}(\cdot)$ in Eq. (7).

3.4 List-wise Loss Function

For each user/query q , we can compute the predicted permutation π , then one direct way to construct $\ell_{\text{list}}(\pi, \pi^*)$ can be formulated as $\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi_i^*, \pi_i)$. However, this formulation is a simple summation of point-wise loss that fails to consider the contextual dependence among items.

Notice that the generation of π_i is conditioned on the arranged items (i.e., $\pi_{<i}$), as shown in Eq. (9). Therefore, the corresponding supervision of each π_i should be $\pi_i^* | \pi_{<i}$, where $\pi^* | \pi_{<i}$ is the

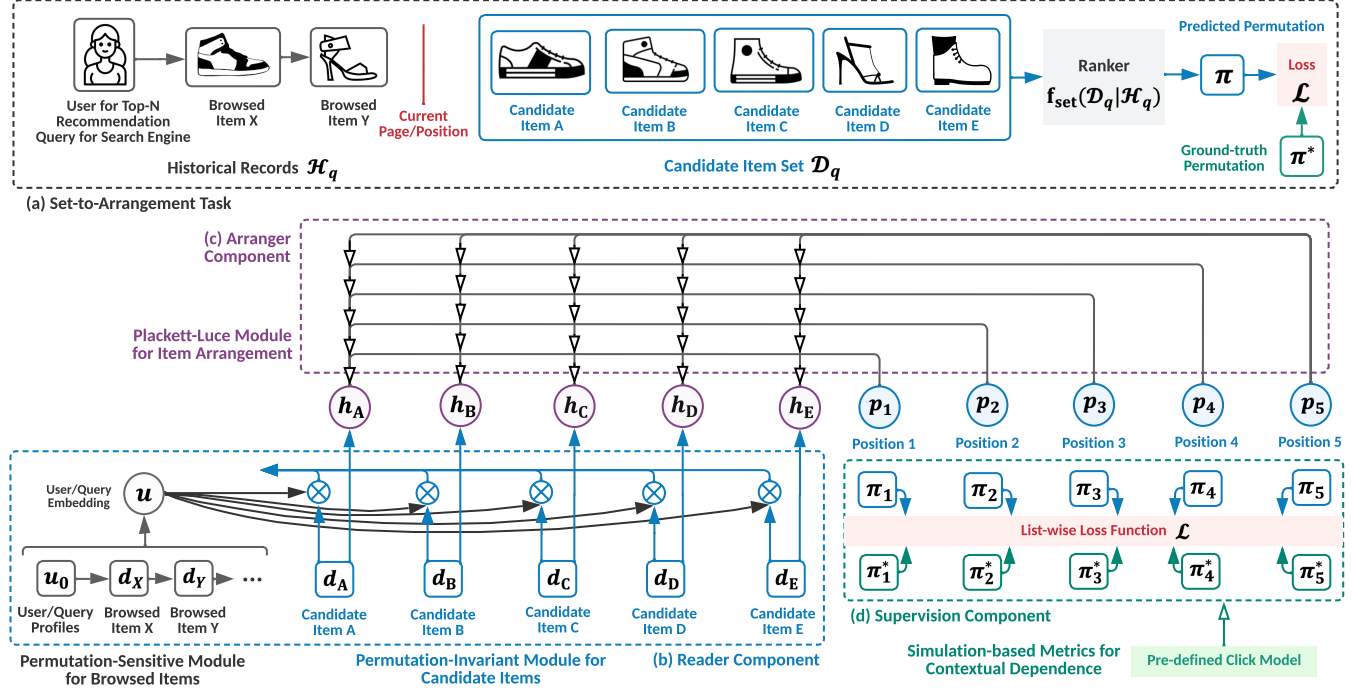


Figure 1: An illustrated example of STARank: (a) the task of STARank is to train a ranker $f_{\text{set}}(\mathcal{D}_q|\mathcal{H}_q)$ which produces the permutation π of the candidate items $\mathcal{D}_q = \{d_A, d_B, d_C, d_D, d_E\}$ in the context of the user q and the browsed items $\mathcal{H}_q = [d_X, d_Y]$. Specifically, (b) STARank reads a set of candidate items into $\{h_A, h_B, h_C, h_D, h_E\}$, where the browsed items together with the user profile are first encoded in a permutation-sensitive fashion, and then are fed into a permutation-invariant module to embed the candidate items. (c) STARank arranges the candidate items into their positions (namely generating the predicted permutation π) according to the attention scores of placing the item into the position that are learned via a Plackett-Luce module. (d) STARank is supervised by optimizing a list-wise loss function $\mathcal{L}$ over π and the oracle permutation π^* where π^* is given. Additionally, we introduce a family of simulation-based ranking metrics. Refer to Sections 2.1, 3.2, 3.3, 3.4 for detailed descriptions of (a), (b), (c), (d).

ground-truth permutation given the items in $\pi_{<i}$ have been arranged, and $\pi_i^*|\pi_{<i}$ is the item at position i in $\pi^*|\pi_{<i}$. In this regard, $\ell_{\text{list}}(\pi, \pi^*)$ can be written as

$$\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi_i^*|\pi_{<i}, \pi_i|\pi_{<i}). \quad (11)$$

However, $\pi^*|\pi_{<i}$ is not available, since we only have the ground-truth permutation π^* . Alternatively, we construct the following surrogate loss to optimize the arrangement of position i when the previous items are arranged following the ground-truth. In this regard, $\ell_{\text{list}}(\pi, \pi^*)$ can be written as

$$\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi_i^*|\pi_{<i}^*, \pi_i|\pi_{<i}^*). \quad (12)$$

However, calculating π_i conditioned on $\pi_{<i}^*$ is still challenging. Therefore, we leverage the internal consistency property of our arranger (i.e., PL models) to derive a computable version of $\ell_{\text{list}}(\pi, \pi^*)$.

We derive the difference between $P(\pi_i|\pi_{<i}^*)$ and $P(\pi_i^*|\pi_{<i}^*)$ as follows.

$$\begin{aligned} P(\pi_i|\pi_{<i}^*) &= \frac{P(\{\pi_i\} \cup \pi_{<i}^*)}{P(\pi_{<i}^*)} = \frac{P(\{\pi_i\} \cup \{\pi_{i-1}^*\} \cup \pi_{<i-1}^*)}{P(\pi_{<i}^*)} \\ &= P(\{\pi_i\} \cup \{\pi_{i-1}^*\}|\pi_{<i-1}^*) \cdot \frac{P(\pi_{<i-1}^*)}{P(\pi_{<i}^*)}, \end{aligned} \quad (13)$$

where $P(\pi_i|\pi_{<i}^*)$ denotes the probability of arranging π_i at position i conditioned on arranged items $\pi_{<i}^*$, and $P(\{\pi_i\} \cup \pi_{<i}^*)$ denotes the probability of arranging $\pi_{<i}^*$ at positions from 1 to $i-1$ and arranging π_i at position i . Eq. (13) tells us the main difference lies in $P(\{\pi_i\} \cup \{\pi_{i-1}^*\}|\pi_{<i-1}^*)$ and $P(\{\pi_i^*\} \cup \{\pi_{i-1}^*\}|\pi_{<i-1}^*)$.

We then formally describe the internal consistency property of PL models in the following lemma.

LEMMA 3.1. *Given a set of items $\mathcal{D}_q$ and one subset $\mathcal{D}'_q \subseteq \mathcal{D}_q$, the internal permutation of the items in $\mathcal{D}'_q$, denoted as π' , is consistent in the context of either $\mathcal{D}_q$ or $\mathcal{D}'_q$, which can be formulated as*

$$P(\pi'|\mathcal{D}_q) = P(\pi'|\mathcal{D}'_q), \quad (14)$$

Algorithm 1: STARank

INPUT: item set $\mathcal{D}_q$, historical data $\mathcal{H}_q$, ground-truth π^* .
OUTPUT: ranker $f_{\text{set}}(\cdot)$, predicted permutation π .

- 1: Initialize all parameters.
- 2: **repeat**
- 3: **for** each user (or query) q **do**
- 4: Read $\mathcal{H}_q, \mathcal{D}_q$ into $\mathbf{u}_q, \{\mathbf{h}_d | d \in \mathcal{D}_q\}$ via Eqs. (3) and (4).
- 5: Recursively arrange $\mathcal{D}_q$ into π using Eq. (7).
- 6: **end for**
- 7: Update parameters by minimizing $\mathcal{L}$ using Eq. (16).
- 8: **until** convergence

where $P(\pi' | \mathcal{D}_q)$ and $P(\pi' | \mathcal{D}'_q)$ are the probabilities of the items in $\mathcal{D}'_q$ satisfying π' in the context of using the candidate item sets $\mathcal{D}_q$ and $\mathcal{D}'_q$ as inputs respectively.

This property is originally proposed in [18], and one can refer to Eq. (27) in [18] for the derivation of LEMMA 3.1 (i.e., Eq. (14)).

Then, by assigning $\mathcal{D}'_q = \{\pi_i, \pi_{i-1}^*\}$ and $\mathcal{D}_q = \{\pi_i\} \cup \pi_{<i}^*$, we can derive that $P(\{\pi_i\} \cup \{\pi_{i-1}^*\}) = P(\{\pi_i\} \cup \{\pi_{i-1}^*\} | \pi_{<i-1}^*)$, namely the relative permutation of π_i and π_{i-1}^* is independent of how positions from 1 to $i-2$ are arranged. In other words, the main difference between $P(\pi_i | \pi_{<i}^*)$ and $P(\pi_i | \pi_{<i}^*)$, as derived in Eq. (13) exists in $P(\pi_i | \pi_{i-1}^*)$ and $P(\pi_i^* | \pi_{i-1}^*)$.

For convenience, let $\pi_0^* = \emptyset$. Then, we re-write Eq. (12) as

$$\ell_{\text{list}}(\pi, \pi^*) = \sum_{i=1}^{|\mathcal{D}_q|} \ell_{\text{point}}(\pi_i | \pi_{i-1}^*, \pi_i^* | \pi_{i-1}^*). \quad (15)$$

Eq. (15) establishes a list-wise approach to optimize STARank by supervising the arrangement at each position. In practice, we formulate the overall loss as

$$\mathcal{L} = - \sum_q \sum_{i=1}^{|\mathcal{D}_q|} \log P(\pi_i = \pi_i^* | \pi_{i-1}^*), \quad (16)$$

where $P(\pi_i = \pi_i^* | \pi_{i-1}^*)$ is the probability of arranging π_i^* after the arranged item π_{i-1}^* .

Compared to existing loss based on the ground-truth relevance scores, $\mathcal{L}$ pushes STARank to fit the ground-truth permutation directly in a position-by-position way, which corresponds to the recursive generation process in our arranger (as shown in Eq. (9)). In other words, $\mathcal{L}$ focuses on identifying and correcting the relative permutations of neighboring pairs that have errors during generating the arrangement.

3.5 Model Analysis

Complexity. From Algorithm 1, we can see that the main components of STARank are the reader component and the arranger component.

Let L denote the length of candidate items of user/query q , then the time complexity of computing an attention score of placing an item is $O(LF_1F_2 + EF_1)$, where E is the number of input features, and F_1 and F_2 are the number of rows and columns of the attention matrix respectively. The above process needs to iterate for each

item and for each user/query. Therefore, the overall complexity is $O(QL^2F_1F_2 + QLEF_1)$ where Q is the number of users/queries.

Property. We theorize that STARank is permutation-invariant regarding the candidate items in the following lemma:

LEMMA 3.2. *For the given item space $\mathcal{D}$, assume that its elements are finite, i.e., $|\mathcal{D}| < \infty$. For each candidate item set $\mathcal{D}_q \subseteq \mathcal{D}$ and its corresponding browsed items $\mathcal{H}_q \subseteq \mathcal{D}$, the ranker $f_{\text{set}}(\mathcal{D}_q | \mathcal{H}_q)$ in Eq. (2) following STARank is permutation-invariant to the elements in $\mathcal{D}_q$.*

PROOF. We first re-express STARank as a function producing the predicted scores for all the possible permutations, namely $f_{\text{set}} : \mathcal{P} \rightarrow \mathbb{R}$ which can be specified as

$$P(\pi | \mathcal{D}_q, \mathcal{H}_q) = \text{PL}(\text{PI}(\mathcal{D}_q, \text{PS}(\mathcal{H}_q))). \quad (17)$$

Here, as $\mathcal{H}_q$ is independent of $\mathcal{D}_q$, we can omit $\text{PS}(\mathcal{H}_q)$ here. As shown in Eq. (6), $\text{PI}(\mathcal{D}_q)$ employing an attention mechanism can be regarded as the summarization of the items in $\mathcal{D}_q$ weighted by the attention scores. And, the formulation of $\text{PL}(\cdot)$ is provided in Eq. (10). According to THEOREM 2 in [39], one can easily conclude that the whole STARank framework is permutation-invariant regarding the candidate items. $\square$

4 EXPERIMENT

4.1 Datasets and Data Pre-processing

Dataset. We introduce 3 industrial recommendation datasets, namely Tmall, Taobao, and Alipay, and 2 learning-to-rank benchmark datasets, namely Yahoo and LETOR in our experiment.

- **Tmall**² is a dataset consisting of 54,925,331 interactions of 424,170 users and 1,090,390 items. These sequential histories are collected by the Tmall e-commerce platform from May 2015 to November 2015 with an average sequence length of 129 and 9 feature fields.
- **Taobao**³ is a dataset containing 100,150,807 interactions of 987,994 users and 4,162,024 items. These user behaviors including several behavior types (e.g., click, purchase, add to chart, item favoring) are collected from November 2007 to December 2007 with an average sequence length of 101 and 4 feature fields.
- **Alipay**⁴ is a dataset collected by Alipay, an online payment application from July 2015 to November 2015. There are 35,179,371 interactions of 498,308 users and 2,200,191 items with an average sequence length of 70 and 6 feature fields.
- **Yahoo**⁵ is a dataset collected by Yahoo, an online search engine. It consists of 29,921 queries and 710k items. Each query document pair is represented by a 700-dimensional feature vector manually assigned with a label denoting relevance at 5 levels [7].
- **LETOR**⁶ is a package of benchmark datasets for research on Learning To Rank, which uses the Gov2 web page collection and two query sets from the Million Query track of TREC2007 and TREC2008. We conduct experiments on MQ2007, one of two

²<https://tianchi.aliyun.com/dataset/dataDetail?dataId=42>

³<https://tianchi.aliyun.com/dataset/dataDetail?dataId=649>

⁴<https://tianchi.aliyun.com/dataset/dataDetail?dataId=53>

⁵<http://webscope.sandbox.yahoo.com>

⁶<https://www.microsoft.com/en-us/research/project/letor-learning-rank-information-retrieval/>

Table 2: Comparison of different rankers on three industrial top-N recommendation datasets in terms of NDCG and MAP at positions 5, 10. * indicates $p < 0.001$ in significance tests compared to the best baseline.

Ranker	Tmall				Alipay				Taobao			
	N@5	N@10	M@5	M@10	N@5	N@10	M@5	M@10	N@5	N@10	M@5	M@10
FM	0.1233	0.1140	0.2494	0.2572	0.1342	0.1277	0.2634	0.2709	0.1124	0.1106	0.2228	0.2354
DeepFM	0.1241	0.1181	0.2515	0.2591	0.1276	0.1214	0.2551	0.2629	0.1117	0.1104	0.2209	0.2339
PNN	0.1248	0.1185	0.2525	0.2602	0.1296	0.1227	0.2601	0.2666	0.1145	0.1123	0.2266	0.2385
LSTM	0.1341	0.1287	0.2660	0.2716	0.1427	0.1348	0.2783	0.2816	0.1308	0.1284	0.2496	0.2587
GRU	0.1311	0.1255	0.2606	0.2668	0.1422	0.1345	0.2773	0.2810	0.1300	0.1275	0.2488	0.2578
DIN	0.1345	0.1305	0.2674	0.2725	0.1401	0.1365	0.2705	0.2751	0.1230	0.1227	0.2351	0.2457
DIEN	0.1243	0.1182	0.2518	0.2594	0.1351	0.1293	0.2632	0.2696	0.1132	0.1122	0.2245	0.2366
SetRank	0.2733	0.2640	0.4090	0.4284	0.2969	0.2850	0.4289	0.4458	0.2594	0.2498	0.3876	0.4062
Seq2Slate	0.2385	0.2340	0.3720	0.3948	0.2452	0.2412	0.3771	0.3993	0.2223	0.2180	0.3437	0.3690
STARank _{PI} ⁻	0.3042	0.3251	0.4095	0.4254	0.3214	0.3264	0.4014	0.4446	0.2632	0.2734	0.3974	0.4214
STARank _{PS} ⁻	0.2952	0.3052	0.3974	0.4035	0.3046	0.3035	0.4046	0.4256	0.2678	0.2742	0.4025	0.4264
STARank	0.3353*	0.3745*	0.4328*	0.4358*	0.4509*	0.4803*	0.5337*	0.5703*	0.3479*	0.4278*	0.4082*	0.4635*
STARank_{PBM}⁺	0.3591*	0.4311*	0.4472*	0.4932*	0.4725*	0.5046*	0.5402*	0.5854*	0.4006*	0.4868*	0.4683*	0.5250*
STARank_{UBM}⁺	0.3482*	0.4208*	0.4359*	0.4828*	0.4481*	0.4709*	0.5189*	0.5644*	0.3930*	0.4792*	0.4545*	0.5164*

Table 3: Comparison of different rankers on Yahoo dataset. * indicates $p < 0.001$ in significance tests.

Ranker	Yahoo			
	N@5	N@10	M@5	M@10
FM	0.2122	0.2434	0.3574	0.3629
DeepFM	0.2483	0.2490	0.3587	0.3662
PNN	0.2563	0.2693	0.3598	0.3602
LSTM	0.2763	0.2731	0.3602	0.3641
GRU	0.2901	0.2876	0.3674	0.3675
DIN	0.2932	0.2975	0.3772	0.3706
DIEN	0.2911	0.2983	0.3703	0.3688
SetRank	0.3053	0.3185	0.3932	0.3868
Seq2Slate	0.3121	0.3234	0.4001	0.3987
STARank _{PI} ⁻	0.3188	0.3210	0.4079	0.4154
STARank _{PS} ⁻	0.3110	0.3356	0.4139	0.4201
STARank	0.3399*	0.3525*	0.4301*	0.4398*
STARank_{PBM}⁺	0.3474*	0.3612*	0.4342*	0.4450*
STARank_{UBM}⁺	0.3355*	0.3543*	0.4324*	0.4463*

Table 4: Comparison of different rankers on LETOR dataset. * indicates $p < 0.001$ in significance tests.

Ranker	LETOR			
	N@5	N@10	M@5	M@10
FM	0.1752	0.1721	0.3282	0.3292
DeepFM	0.1746	0.1689	0.3243	0.3204
PNN	0.1775	0.1739	0.3296	0.3301
LSTM	0.1851	0.1795	0.3405	0.3384
GRU	0.1739	0.1741	0.3170	0.3205
DIN	0.1550	0.1671	0.2815	0.2988
DIEN	0.1818	0.1811	0.3409	0.3402
SetRank	0.2035	0.2246	0.4045	0.4002
Seq2Slate	0.2436	0.2546	0.4122	0.4031
STARank _{PI} ⁻	0.2443	0.2568	0.4172	0.4075
STARank _{PS} ⁻	0.2467	0.2606	0.4231	0.4123
STARank	0.2824*	0.2622*	0.4876*	0.4529*
STARank_{PBM}⁺	0.2843*	0.2646*	0.4906*	0.4573*
STARank_{UBM}⁺	0.2863*	0.2668*	0.4887*	0.4587*

datasets for supervised learning-to-rank tasks in LETOR. MQ2007

contains 2,476 queries and 85K documents. Each query-document pair is represented by a 46-dimensional feature vector with a manually assigned 3-level label [28].

Data Pre-processing⁷. We split each dataset using the timestep. Let T denote the length of user browsing logs. We first filter out the data instances whose browsing logs are smaller than 30 (i.e., $T < 30$). Next, for each user q , we assign 1-st to $(T-20)$ -th items as the training dataset where 1-st to $(T-30)$ -th items are used as the user’s history data (i.e., $\mathcal{H}_q$) and $(T-29)$ -th to $(T-20)$ -th items construct the candidate item set (i.e., $\mathcal{D}_q$). In validation, we use 1-st to $(T-20)$ -th items as $\mathcal{H}_q$ and the $(T-19)$ -th to $(T-10)$ -th items as $\mathcal{D}_q$; while for evaluation, we use 1-st to $(T-10)$ -th items as $\mathcal{H}_q$ and $(T-9)$ -th to T -th items as $\mathcal{D}_q$. For $\mathcal{H}_q$ and $\mathcal{D}_q$, we keep their original browsing orders and do not adopt any initial ranker to generate the initial ranking list.

4.2 Experimental Configurations

Simulation-based Ranking Metrics. Consider that real-world datasets usually offer ground-truth relevance scores for candidate items, instead of ground-truth permutations. Therefore, we are required to first generate the ground-truth permutations. For this purpose, we introduce a ranking metric, denoted as $R(\cdot)$. Let $\mathcal{P}$ denote the set of all the possible permutation of $\mathcal{D}_q$. Then, we have $|\mathcal{P}| = A_{|\mathcal{D}_q|}^{|\mathcal{D}_q|} = O(|\mathcal{D}_q|!)$ where $A_{|\mathcal{D}_q|}^{|\mathcal{D}_q|}$ represents permutation operator. After we enumerate all the possible permutations $\pi \in \mathcal{P}$, we select the one received the highest score in terms of $R(\cdot)$ as the oracle permutation π^* . In other words, we have:

$$\pi^* = \arg \max_{\pi \in \mathcal{P}} R(\pi). \quad (18)$$

We note that conventional ranking metrics, such as NDCG, only consider the effects of the positions of the arranged items but overlook the contextual dependence among the items (i.e., the probability of

⁷We note that our setting on how to divide datasets is different from previous work such as SetRank [25] which would make the results not consistent with their results reported in original papers, because our framework requires a sequence of items as the history (i.e., $\mathcal{H}_q$) and another set of items as the candidate items (i.e., $\mathcal{D}_q$).

Table 5: Comparison of different rankers on three industrial top-N recommendation datasets in terms of the proposed simulation-based ranking metrics based on PBM and UBM at positions 5, 10. * indicates $p < 0.001$ in significance tests compared to the best baseline.

Ranker	Tmall				Alipay				Taobao			
	P@5	P@10	U@5	U@10	P@5	P@10	U@5	U@10	P@5	P@10	U@5	U@10
FM	0.1026	0.1027	0.1015	0.1016	0.1138	0.1157	0.1013	0.1014	0.1073	0.1067	0.1013	0.1017
DeepFM	0.1023	0.1023	0.1005	0.1008	0.1037	0.1040	0.1008	0.1009	0.1093	0.1097	0.1008	0.1015
PNN	0.1248	0.1185	0.1014	0.1016	0.1023	0.1021	0.1002	0.1005	0.1142	0.1155	0.101	0.1016
LSTM	0.2074	0.2734	0.1301	0.1460	0.2137	0.2068	0.1274	0.1488	0.2498	0.2922	0.2025	0.2842
GRU	0.2188	0.3066	0.1299	0.1437	0.2292	0.2208	0.1308	0.1512	0.2541	0.3023	0.1928	0.2747
DIN	0.2725	0.2474	0.2493	0.1818	0.2257	0.2333	0.1443	0.1648	0.2748	0.2741	0.2099	0.2846
DIEN	0.2520	0.3693	0.1014	0.1012	0.2366	0.2194	0.1281	0.1456	0.2345	0.2986	0.1891	0.2654
SetRank	0.3778	0.3973	0.3378	0.3650	0.4171	0.4330	0.3346	0.3584	0.3094	0.3191	0.3704	0.3820
Seq2Slate	0.3182	0.3455	0.3210	0.3482	0.3337	0.3600	0.3254	0.3533	0.3285	0.3541	0.3189	0.3470
STARank _{PI}	0.3962	0.3967	0.4346	0.4426	0.5025	0.5046	0.4325	0.4602	0.3429	0.3478	0.4385	0.4255
STARank _{PS}	0.3873	0.3863	0.4253	0.4371	0.5010	0.5024	0.4242	0.4523	0.3321	0.3368	0.4257	0.4211
STARank	0.4240*	0.4010*	0.4435*	0.4493*	0.5283*	0.5058*	0.4578*	0.4888*	0.3529*	0.3599*	0.4418*	0.4308*

a user favoring an item should be dependent on other items posted in the same sequence).

To this end, we propose to construct a virtual user to consider the contextual dependence and make the evaluation according to the user’s feedbacks. These virtual user models are known as the click models [9], which generate users’ feedback by making some reasonable assumptions based on some heuristics or eye-tracking experimental results [22]. Formally, we use CM to denote an arbitrary click model (CM) (e.g., position-based model (PBM) [31], user browsing model (UBM) [12]) treated as a black block to generate the user’s feedback for the input sequence. Then, we introduce a new family of the ranking metrics based on the aforementioned simulations (i.e., click models), which is called simulation-based ranking metrics and defined as

$$R_{CM}(\pi) = \sum_{i=1}^{|\mathcal{D}_q|} P(c_{\pi_i} | \pi_{<i}; CM) = \sum_{i=1}^{|\mathcal{D}_q|} P(o_{\pi_i} | \pi_{<i}; CM) \cdot P(r_{\pi_i} | \pi_{<i}; CM), \quad (19)$$

which reflects a simple fact that a user clicks an item d (i.e., $P(c_d = 1)$) only when it is both observed (i.e., $P(o_d = 1)$) and perceived as relevant (i.e., $P(r_d = 1)$), and the observation probability and the relevance probability are determined by the click model CM. We use PBM and UBM as examples for CM and introduce their detailed configurations as follows. **PBM** [31] simulates user browsing behavior based on the assumption that the bias of a document only depends on its position, which can be formulated as $P(o_i) = \rho_i^\tau$, where ρ_i represents position bias at position i and $\tau \in [0, +\infty]$ is a parameter controlling the degree of position bias. The position bias ρ_i is obtained from an eye-tracking experiment in [22] and the parameter τ is set as 1 by default. It also assumes that a user decides to click a document d_i according to the probability $P(c_i) = P(o_i) \cdot P(r_i)$. **UBM** [12] is an extension of the PBM model that has some elements of the cascade model. The examination probability depends not only on the rank of an item d_i but also on the rank of the previously clicked document $d_{i'}$ as $P(o_i = 1 | c_{i'} = 1, c_{i'+1} = 0, \dots, c_{i-1} = 0) = \gamma_0$. Similarly, we get γ_0 from the eye-tracking experiments in [12, 22]. The click probability is determined by $P(c_i) = P(o_i) \cdot P(r_i)$.

In this regard, NDCG metric is a special case of the proposed ranking metrics using PBM as CM. Concretely, the formulation of NDCG can be written as

$$R_{NDCG}(\pi) = \frac{1}{N} \sum_{i=1}^{|\mathcal{D}_q|} \left(\frac{1}{\log_2(i+1)} \right) (2^{r_{\pi_i}} - 1), \quad (20)$$

where N normalizes for the number of relevant items, and r_{π_i} is the relevance of item π_i . By comparing Eq. (20) to Eq. (19), we reveal that $R_{NDCG}(\cdot)$ is a special case of $R_{CM}(\cdot)$ by assigning

$$\begin{aligned} P(o_{\pi_i} | \pi_{<i}; CM) &= \frac{1}{N_o} \left(\frac{1}{\log_2(i+1)} \right), \\ P(r_{\pi_i} | \pi_{<i}; CM) &= \frac{1}{N_r} (2^{r_{\pi_i}} - 1), \end{aligned} \quad (21)$$

where $N = N_o \cdot N_r$. From Eq (21), we can see that the observation probability of an arbitrary item is solely determined by its position, showing that CM here is a special case of PBM.

Noting that STARank metrics can be seamlessly adopted in the supervision generation, we can re-formulate Eq. (18) as

$$\pi^* = \arg \max_{\pi \in \mathcal{P}} R_{CM}(\pi). \quad (22)$$

Note that the supervisions of STARank are determined by the ranking metrics. For those datasets described in with binary scores in the browsing logs, there might exist multiple oracle permutations when applying either Eq. (18) or (22). In these cases, we randomly choose one oracle permutation as the supervision. Also, these cases would occur during the evaluation phase, where we act similarly. We notice that there is a related paper [33] investigating the impacts of different evaluation scheme regarding the same scores, which is out of the scope of this paper and we leave it for future work.

Hyperparameter Setting. The learning rate is decreased from the initial value 1×10^{-2} to 1×10^{-6} during the training process. The batch size is set as 100. The weight for the L2 regularization term is 4×10^{-5} . The dimension of embedding vectors is set as 64. The dropout rate is 0.5. All experimental comparisons are conducted with 10 different random seeds. All the models are trained under the same hardware settings with 6-Core AMD Ryzen 9 5950X (2.194GHZ), 62.78GB RAM, NVIDIA GeForce RTX 3080 cards.

All code will be released at publication time.

4.3 Baseline Methods and Ranking Metrics

Baseline Descriptions. We compare STARank against 9 strong baselines, introduced as follows. **FM** [30] is the factorization machine using the linear projection and inner product of features to measure the user-item relevance. **DeepFM** [15] is a generalized model consisting of a FM as a wide component and a MLP as a deep component. **PNN** [29] is product-based neural networks including an embedding layer and a product layer to model user-item interactions. **DIN** [41] designs a local activation unit to capture user interests from historical records. **LSTM** [17] is a standard long short memory approach widely used for modeling user’s sequential pattern. **GRU** [16] uses the gate recurrent units to model sequential user behaviors. **DIEN** [40] is an extension of DIN that builds an interest extractor layer to model user’s temporal interests. **SetRank** [25] uses a stack of self-attention blocks based on set transformer [23] to model the cross-item interactions. **Seq2Slate** [3] employs the pointer network [27] to sequentially encode previously selected items and predict the next one. All the methods are trained based on click feedback only.

Evaluation Metrics. We evaluate the performance of these methods according to two series of ranking metrics, namely conventional ranking metrics and simulation-based ranking metrics built upon the specific click models. To be more specific, for the former ones, we choose MAP (Mean Average Precision) and NDCG (Normalized Distributed Cumulative Gain) at position 5, 10, denoted as M@K and N@K where K is 5, 10. For the later ones, we use PBM and UBM as the click models and compute the cumulative click probability at positions 5, 10, which are denoted as P@K and U@K and K is 5, 10.

Considering that the implementation of STARank is conditioned on which click model we choose. Here, besides the standard implementation of STARank using NDCG as the metric, we further introduce two variants using PBM and UBM as the click models:

- **STARank** is the proposed framework, which uses NDCG over the whole ranking list to compute π^* following Eq. (18).
- **STARank^{+PBM}** is a variant of STARank using $R_{PBM}(\cdot)$ in Eq. (19) to produce π^* (i.e., PBM [31] as the click model).
- **STARank^{+UBM}** is another variant of STARank using $R_{UBM}(\cdot)$ in Eq. (19) to generate π^* (i.e., UBM [12] as the click model).

We also develop the following variants to investigate the impact of each component of the proposed framework:

- **STARank^{-PI}** is a variant of STARank using multi-layer perceptron (MLP) layer as PI module to encode the input items instead of the attention network, where the user representation is concatenated with item embeddings to be fed into the MLP layer.
- **STARank^{-PS}** is a variant of STARank using the MLP layer as PS module to encode the sequential records instead of the LSTM.

4.4 Performance Comparison

Tables 2, 3, 4 and 5 summarize the comparison results on the top-N recommendation datasets and learning-to-rank datasets, in terms of conventional ranking metrics and simulation-based ranking metrics. Our major findings are listed as follows.

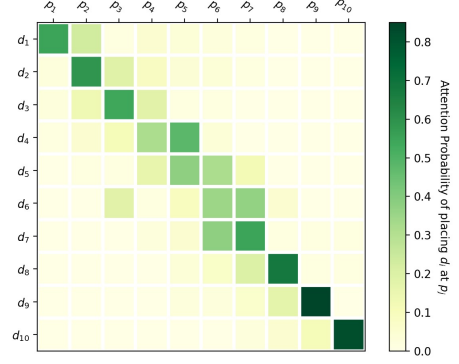


Figure 2: Visualization of attention probabilities of placing item d_i at position p_j on Alipay dataset.

STARank consistently outperforms all the baselines scoring and sorting. From these tables, we can clearly see that STARank outperforms all these baselines including classical tower-based methods (e.g., DeepFM), sequential methods (e.g., DIEN), and recently proposed set-to-sequence methods (e.g., SetRank) in terms of all the conventional and new proposed simulation-based ranking metrics. Since all these baselines are supervised by the relevance score of each item instead of directly optimizing the model by the permutation of the items in an end-to-end fashion, these results would indicate the superiority of the overall design of the proposed framework.

STARank works well on the proposed simulation-based metrics. From comparisons between Tables 2 and 5, we can see that there are larger performance gains from STARank in terms of the simulation-based metrics than the conventional configurations. A possible explanation is that compared to the conventional ranking metrics, the new proposed simulation-based configurations consider the effects from the contextual dependence; and thus supervising the permutations of the ranking list would perform better in this case.

SetRank and Seq2Slate consistently outperforms the other baselines. From these results, we also can observe that among 9 baseline methods, SetRank and Seq2Slate can consistently achieve the best performance. One possible reason is that due to our experimental configuration typically assigns a set of items served as the historical records, then these methods can effectively leverage the contexts to better capture the user needs.

4.5 Ablation Studies

Impact of Permutation-Invariant Module. To investigate the performance gain from our permutation-invariant module, we evaluate the performance of STARank^{-PI} and report the corresponding results in Tables 2, 3, 4, and 5. From comparisons between STARank and STARank^{-PI}, the results demonstrate the superiority of our proposed attention networks introduced in Section 3.2 which incorporates user representation to produce the representation vector for each item.

Impact of Permutation-Sensitive Module. We use STARank^{-PS} to show the performance changes by replacing our RNN module

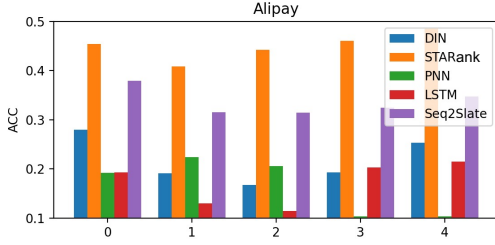


Figure 3: Performance comparisons of STARank against baselines in terms of accuracy at top 5 positions on Alipay dataset.

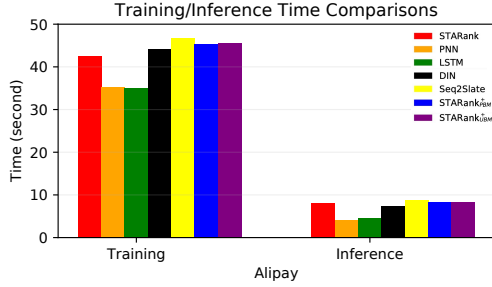


Figure 4: Comparisons of training and inference time of STARank and its variants against baselines on Alipay dataset.

with the MLP layer which is a permutation-invariant module and is not suitable to encode the permutation-sensitive user historical data. Results summarized in Tables 2, 3, 4, and 5 verify the performance gain by using a PS module to model the user’s history.

Impact of Supervision Generation. As described in the experimental configuration, the proposed simulation-based ranking metrics can not only be used for evaluation but also can be adopted in the ground-truth permutation generation (i.e., Eq. (22)). As Table 5 shows the experimental results of each model under these simulation-based ranking metrics, we further form the variants STARank⁺_{PBM} and STARank⁺_{UBM} whose supervision generations are simulation-based but evaluations are under conventional ranking metrics. From the corresponding results shown in Table 2, we can see that PBM-based supervisions consistently improve the ranking performance, while UBM-based supervisions succeed in Tmall and Taobao datasets and fail in Alipay dataset. One possible explanation is that these supervision signals encourage the model to encode the sequence dependence based on the assumptions of certain click models. Therefore, if these assumptions hold in the dataset, the ranking performance would be improved; otherwise, the ranking performance would be hurt.

4.6 In-depth Analysis

Visualization Analysis. To better understand the behavior of the model, we visualize the probabilities of the attention scores from Eq. (9), which shows the average probability of placing item d_i (corresponding to d in Eq. (9)) at position p_j (corresponding to π_i in Eq. (9)) at each step. As Figure 2 depicts, the model produces the permutations that are close to the input order, but with the some items are placed to the neighboring positions. A possible reason is that the input orders provided by Taobao platform are expected to

achieve a good performance; and our method in this case can be regarded as a re-ranking model to refine the input orders.

Also, these attention values provide another perspective to explain why our framework performs better than existing ranking methods following PRP. As revealed in [10], considering there are multiple bias in ranking, the maximization of the utility can be seen as solving the maximum-weight matching on the item-position bipartite graph, where the edge weight between an item and a position denotes the utility of placing the item at this position. In this regard, our attention scores could be the estimation of the utility. In contrast, PRP learning to assign individual score to each item, does not learn to estimate these the edge weights.

Accuracy at Each Position. In order to precisely investigate the model performance at each position, we compute the accuracy (ACC) of the ranking list provided by PNN, LSTM, DIN, Seq2Slate, and STARank, and report the corresponding results on Alipay dataset in Figure 3. Given the predicted permutation π and the oracle permutation π^* , for each data instance, ACC value equals 1 at position i when $\pi_i = \pi_i^*$ holds; 0 otherwise. From Figure 3, we can see that our method can consistently outperform all these baseline methods at all the positions. We also note that ACC values at the highest and lowest positions are higher than other positions. One reasonable explanation is that the items that a user extremely favors or dislikes are much easier to distinguish than other items.

4.7 Complexity Analysis

We investigate the time complexity of STARank, STARank⁺_{PBM}, and STARank⁺_{UBM}. For comparisons, we also include the baseline methods PNN, LSTM, DIN, and Seq2Slate. Figure 4 depicts the training and inference times for one round of the whole data. From Figure 4, we can see that STARank⁺_{PBM} and STARank⁺_{UBM} are less computationally efficient than STARank, as these variants need to apply the click models to generate the ground-truth permutations. We also note that the time complexity of STARank and its variants are comparable to the other baselines.

4.8 Robustness Analysis

We evaluate the robustness of STARank under different amounts of training data in Alipay dataset. For comparison, we also employ LSTM as a baseline method. Results are reported in Figure 5, which demonstrate that after reaching a threshold (60% in this case), STARank is almost as robust as LSTM regarding the amounts of training data.

5 CONCLUSION AND FUTURE WORK

In this paper, we present a novel set-to-arrangement framework named STARank to directly generate the permutations for input sets of items without the needs of scoring and sorting items. We emphasize that our contributions lie in both the architecture and supervision designs, where the former one allows the model to encode the candidate item set in a permutation-invariant fashion in the context of the permutation-sensitive historical data; and the latter one provides a computational advantage of directly supervising the proposed framework by ground-truth permutations in an end-to-end manner. These contributions together enable STARank to be fully differentiable, and also allow STARank to operate when

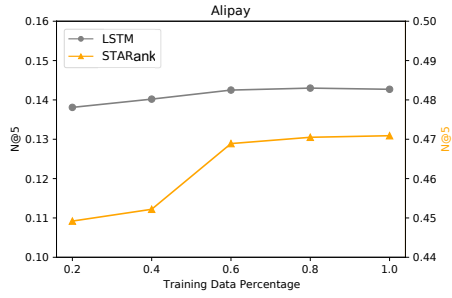


Figure 5: Performance change of STARank against LSTM with different amounts of training data on Alipay dataset.

only ground-truth permutations instead of ground-truth relevance scores for items are accessible. We are also aware that conventional ranking metrics (e.g., NDCG) do not consider the contextual dependence, and propose a new family ranking metrics based on the simulations. Our experimental results demonstrate that our approach can consistently deliver significant improvements in terms of both conventional and the proposed ranking metrics. For future work, it would be interesting to investigate possible applications of the proposed framework in the real-world e-commerce platforms.

REFERENCES

- [1] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *SIGIR*.
- [2] Leif Azzopardi, Paul Thomas, and Nick Craswell. 2018. Measuring the utility of search engine result pages: an information foraging based measure. In *SIGIR*.
- [3] Irwan Bello, Sayali Kulkarni, Sagar Jain, Craig Boutilier, Ed Chi, Elad Eban, Xiyang Luo, Alan Mackey, and Ofer Meshi. 2018. Seq2slate: Re-ranking and slate optimization with rnns. *arXiv preprint arXiv:1810.02019* (2018).
- [4] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. Learning to rank using gradient descent. In *ICML*.
- [5] Christopher JC Burges. 2010. From ranknet to lambdarank to lambdamart: An overview. *Learning* (2010).
- [6] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In *ICML*.
- [7] Olivier Chapelle and Yi Chang. 2011. Yahoo! learning to rank challenge overview. In *Proceedings of the learning to rank challenge*. 1–24.
- [8] Olivier Chapelle, Donald Metzler, Ya Zhang, and Pierre Grinspan. 2009. Expected reciprocal rank for graded relevance. In *CIKM*.
- [9] Aleksandr Chuklin, Ilya Markov, and Maarten de Rijke. 2015. Click models for web search. *Synthesis lectures on information concepts, retrieval, and services* 7, 3 (2015), 1–115.
- [10] Xinyi Dai, Jiawei Hou, Qing Liu, Yunjia Xi, Ruiming Tang, Weinan Zhang, Xiuqiang He, Jun Wang, and Yong Yu. 2020. U-rank: Utility-oriented learning to rank with implicit feedback. In *CIKM*.
- [11] Yajuan Duan, Long Jiang, Tao Qin, Ming Zhou, and Heung Yeung Shum. 2010. An empirical study on learning to rank of tweets. In *Proceedings of the 23rd International Conference on Computational Linguistics (Coling 2010)*.
- [12] Georges E Dupret and Benjamin Piwowarski. 2008. A user browsing model to predict search engine click data from past observations. In *SIGIR*.
- [13] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of statistics* (2001), 1189–1232.
- [14] John Guiver and Edward Snelson. 2009. Bayesian inference for Plackett-Luce ranking models. In *proceedings of the 26th annual international conference on machine learning*. 377–384.
- [15] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. In *IJCAI*.
- [16] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *ICLR* (2015).
- [17] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [18] David R Hunter. 2004. MM algorithms for generalized Bradley-Terry models. *The annals of statistics* (2004).
- [19] Thorsten Joachims. 2002. Optimizing search engines using clickthrough data. In *KDD*.
- [20] Thorsten Joachims. 2006. Training linear SVMs in linear time. In *KDD*.
- [21] Thorsten Joachims, Laura Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2017. Accurately interpreting clickthrough data as implicit feedback. In *SIGIR*.
- [22] Thorsten Joachims, Laura A Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2005. Accurately interpreting clickthrough data as implicit feedback. In *SIGIR*.
- [23] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiosek, Seungjin Choi, and Yee Whye Teh. 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In *ICML*.
- [24] Tie-Yan Liu. 2011. Learning to rank for information retrieval. (2011).
- [25] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. Setrank: Learning a permutation-invariant ranking model for information retrieval. In *SIGIR*.
- [26] Robin L Plackett. 1975. The analysis of permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* (1975).
- [27] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. 2017. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *CVPR*.
- [28] Tao Qin, Tie-Yan Liu, Jun Xu, and Hang Li. 2010. LETOR: A benchmark collection for research on learning to rank for information retrieval. *Information Retrieval* (2010).
- [29] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-based neural networks for user response prediction. In *ICDM*.
- [30] Steffen Rendle. 2010. Factorization machines. In *ICDM*.
- [31] Matthew Richardson, Ewa Dominowska, and Robert Ragno. 2007. Predicting clicks: estimating the click-through rate for new ads. In *WWW*.
- [32] Stephen E Robertson. 1977. The probability ranking principle in IR. *Journal of documentation* (1977).
- [33] Zhiqing Sun, Shikhar Vashishth, Soumya Sanyal, Partha Talukdar, and Yiming Yang. 2019. A re-evaluation of knowledge graph completion methods. *ACL* (2019).
- [34] Michael Taylor, John Guiver, Stephen Robertson, and Tom Minka. 2008. Softrank: optimizing non-smooth rank metrics. In *WSDM*.
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NIPS*.
- [36] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer networks. *arXiv preprint arXiv:1506.03134* (2015).
- [37] Fen Xia, Tie-Yan Liu, Jue Wang, Wensheng Zhang, and Hang Li. 2008. Listwise approach to learning to rank: theory and algorithm. In *ICML*.
- [38] Emine Yilmaz, Manisha Verma, Nick Craswell, Filip Radlinski, and Peter Bailey. 2014. Relevance and effort: An analysis of document utility. In *CIKM*.
- [39] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. 2017. Deep sets. *arXiv preprint arXiv:1703.06114* (2017).
- [40] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *AAAI*.
- [41] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *KDD*.